CS 0007: Introduction to Java Lecture 13

Nathan Ong
University of Pittsburgh
October 25, 2016

Announcements

- Exam grades are posted on Courseweb
- You can see your exam during office hours or make an appointment, but you may not keep them
- Project 0 due October 31, 2016 at 11:59 PM via submission link on Courseweb
- Project 1 released soon

Usage

- When do I use For Loops?
  - Iteration, or going through all of the elements of an array or other type of list.
  - When the number of times you need to loop is known, either explicitly with a value, or through a variable.
- While loops are used otherwise

Multi-dimension Arrays

- Arrays don't just have to be lists
- They can also be grids, cubes, hyper-cubes, ...

```
Type[]...[] name = new Type[size1]...[sizeN];
```

```
Type[]...[] name = values...s;
```

```
int[][] listList = {{1,2,3},
    {4,5,6},{7,8,9}};
```

Things to Remember

- In 2-D Arrays, referencing an element requires TWO PAIRS of square brackets
- To get the #1 from listList
- **`listList[0][0]`**
- To get the #4 from listList
- **`listList[1][0]`**
- Remember: element listList[row][col]

How Do We Go Through a 2-D Array?

- If it took one for loop to go though a 1-D array, then...
- It will take two for loops to go through a 2-D array
- It will take N for loops to go through an N-D array
- If you don't have to go through everything, don't. It just wastes time

listList[row][col]

Let's Test It on listList

```
public class Test
{
     public static void main(String[] args)
     {
         int[][] list = {{1,2,3},{4,5,6},{7,8,9}};
         for(int i = 0; i < list.length; i++)
         {
             for(int j = 0; j < list[i].length; j++)
             {
                 System.out.println("Counting...
                     Now at " + list[i][j]);
             }//end inner-loop for(j < list[i].length)
         }//end outer-loop for(i < list.length)
     }//end method main
}//End class Test
```

For convenience sake, listList list

What is the output?

Good Coding Practices

- Whenever you make any variable or array, ensure that you fill it with the appropriate values, or use some default non-important value.
- Rule of thumb:
  - Numbers/`char` → 0(.0)
  - `boolean` → `false`
  - Objects → `null`
  - Arrays → Depends on type, but every element should be set to something explicitly

Meaning of `null`

- Null indicates the variable refers to nothing
- There is no actual object with the given variable name
- As expected, you cannot do anything useful with a variable set to `null`
- Checking for `null` (in an `if`-statement or other `boolean` expression) uses `==`

```
Scanner myScan = new Scanner(System.in);
if(myScan == null)
{
  System.err.println("Something wrong happened");
}//clearly never executed
```

Recursion

- Recursion is the process of a function (or set of functions) that calls itself.
- Recursion is really just a special name for calling the same function within itself
- Recursion ends at a base case.

A Simple Example

- I want to compute n! (factorial)

n! = n * (n-1) * (n-2) * … * 1

- I can easily do this with loops, but for the sake of an easy example, we can easily break this down

A Simple Example

- I want to compute n! (factorial)

n! = n * (n-1) * (n-2) * … * 1

- Recursion is the breakdown of a problem into smaller chunks, like Matryoshka dolls



Source: https://upload.wikimedia.org/wikipedia/commons/7/71/Russian-Matroshka.jpg

Recursive Factorial

- How can we take the large problem and break it up into something smaller?
- Can we build up a solution?
- With factorial, it is easy to see how to break this down by this equivalence:

  n! = n * ((n-1) * (n-2) * ... * 2 * 1)

  = n * (n-1)!
- To compute n!, we can rely on (n-1)!

How Do We Put This In Code?

- We already know by the definition of recursion that we have to call the function inside of itself.

```
public static int factorial(int n)
{
  factorial(<something>);
  return <something>;
}//end function(int)
```

Recursive Factorial

- Let us use the knowledge that we know from the equivalence.

  $n! = n * ((n-1) * (n-2) * \ldots * 2 * 1)$

  $= n * (n-1)!$

- In order to return n!, we need to know the value of (n-1)! and then multiply it with n.

How Do We Put This In Code?

- In order to return n!, we need to know the value of (n-1)! and then multiply it with n.

```
public static int factorial(int n)
{
        factorial(<something>);
        return <something>;
}//end function(int)
```

How Do We Put This In Code?

- In order to return n!, we need to know the value of (n-1)! and then multiply it with n.

```
public static int factorial(int n)
{
        factorial(<n-1>);
        return <n*(n-1)!>;
}//end function(int)
```

How Do We Put This In Code?

- In order to return n!, we need to know the value of (n-1)! and then multiply it with n.

```
public static int factorial(int n)
{
      int prevFact = factorial(n-
  1);
      return n*prevFact;
}//end function(int)
```

How Do We Put This In Code?

- In order to return n!, we need to know the value of (n-1)! and then multiply it with n.

```
public static int factorial(int n)
{
      return n*factorial(n-1);
}//end function(int)
```

How Do We Put This In Code?

- If we just use this, we run into the problem that this goes on forever.
- When do we stop? When is the base case?

```
public static int factorial(int n)
{
  return n*factorial(n-1);
}//end function(int)
```

How Do We Put This In Code?

- When n <= 1, n! is just 1

```
public static int factorial(int n)
{
  if(n <= 1)
  {
  return 1;
  }
  return n*factorial(n-1);
}//end function(int)
```

How Do We Put This In Code?

```
public class FactorialTester
{
     ...(Function)
     public static void main(String[] args)
     {
     Scanner scanner = new Scanner(System.in);
     System.out.println("Enter an integer");
     int n = scanner.nextInt();
     System.out.println(n + "! = " +
     factorial(n));
     }//end method main(String[])
}//End class FactorialTester
```

OBJECT ORIENTED PROGRAMMING

Revisiting the History of Languages

- Assembly – Low-level hardware based code
- Structured – Introduces subroutines (i.e. functions) and well-defined looping structures
- Object-Oriented – Introduces classes and a philosophy about programming

It’s In the Name

- Object-Oriented Programming is as it sounds. Assume everything can be modeled as an object.

Terminology

- A class is code that describes objects of that type
- An instance is a particular object of a given type
- A field is a property of an object
- A method is a function that is provided by the class for an object of the class type

Static

- Referenced by keyword `static`
- Property or method is not dependent on the individual instantiated object, but rather the class as a whole
- The property/method exists between instances; any change made to static fields are reflected through all instances of the class
- Static methods can only manipulate static fields

Main Portions of OOP

- Building a class
- Determining inter-class relationships
  - Subclasses
  - Superclasses
  - Ownership